

COMPREHENSIVE PENETRATION TESTING REPORT & CAPSTONE

Enterprise-Style Vulnerability Assessment and Penetration Testing (VAPT)

Prepared by
Drushya ND
Computer Science and Engineering
Canara Engineering College

Training Program
RabTech Academy — Cybersecurity & Ethical Hacking

Assessment Date: 02 September 2026
Classification: Training / Academic Lab Report

1. Document Control & Assessment Statement

Report ID: RTA-VAPT-2026-01

Assessment Type: Training / academic penetration testing and vulnerability assessment

Primary Lab Target: OWASP Juice Shop / intentionally vulnerable web application

Assessment Standard: OWASP Testing Guide + OWASP Top 10 mapping

Scoring: CVSS v3.1 Base Scores with contextual prioritization

Status: Final submission draft for mentor review

Authorization & ethics: Testing described in this report is intended for authorized training environments and deliberately vulnerable applications. No claim is made that the findings represent an unauthorized test of a third-party production system.

2. Executive Summary

The capstone consolidates application-security testing, vulnerability analysis, secure authentication development, and security-tooling practice into a single VAPT-style report.

Overall assessment: the training application demonstrates several high-impact vulnerability classes representative of real-world web application risk. Injection and authentication weaknesses receive the highest remediation priority, followed by sensitive-data exposure, security misconfiguration, client-side XSS, and object-level authorization.

CVSS is used to communicate vulnerability severity, while remediation priority also considers exploitability, data sensitivity, business impact, exposure, and compensating controls.

3. Scope, Objectives & Rules of Engagement

Scope includes web application security, authentication, authorization, configuration review, vulnerability-management prioritization and secure-development validation.

Out of scope: destructive testing against real systems, denial-of-service activity against public infrastructure, persistence, credential theft from real users, malware deployment, and unauthorized access to third-party assets.

Success criteria: produce a formal 15–20 page report, map findings to OWASP, provide CVSS 3.1 vectors, define remediation timelines and prepare a security-lab repository structure.

4. Methodology & Testing Approach

Workflow: planning → reconnaissance → enumeration → validation → risk analysis → remediation → retesting.

Evidence quality principle: each finding should contain the affected component, precondition, safe reproduction description, impact, severity, remediation and retest criteria.

5. Asset Inventory & Attack Surface

Assets considered include the web application, authentication service, database layer, browser client, deployment/configuration, security tooling and source repository.

Attack-surface themes include user-controlled input, authentication endpoints, object identifiers, client-side rendering, sensitive-data paths, configuration metadata and dependency/runtime behavior.

6. Risk Matrix & Prioritization

ID	Finding	Severity	CVSS	Target SLA
VAPT-01	Injection	Critical	9.8	Immediate
VAPT-04	Authentication	Critical	9.1	Immediate
VAPT-05	Sensitive Data	High	8.2	≤ 7 days
VAPT-06	Misconfiguration	High	7.3	≤ 7 days
VAPT-02	DOM XSS	High	6.1	≤ 7 days
VAPT-03	Access Control	Medium	6.3	≤ 14 days

7. CVSS v3.1 Scoring Method

CVSS v3.1 communicates vulnerability severity through Base, Temporal and Environmental metric groups. This report uses Base Scores and vectors so that the severity decision is transparent and reproducible.

Metric	Purpose
AV	How the vulnerable component can be reached.
AC	Whether exploitation requires special conditions.
PR	Privileges required before exploitation.
UI	Whether another user must take an action.
S	Whether impact crosses a security authority boundary.
C/I/A	Confidentiality, Integrity and Availability impact.

8.1 Technical Finding — VAPT-01

Field	Assessment result
Finding	Injection — SQL/NoSQL Injection Risk
Severity	Critical
CVSS v3.1	9.8
Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
OWASP mapping	A03:2021 — Injection

Description: Application input was observed to be trusted by backend query logic in the deliberately vulnerable training application, creating a path for attacker-controlled input to influence database operations.

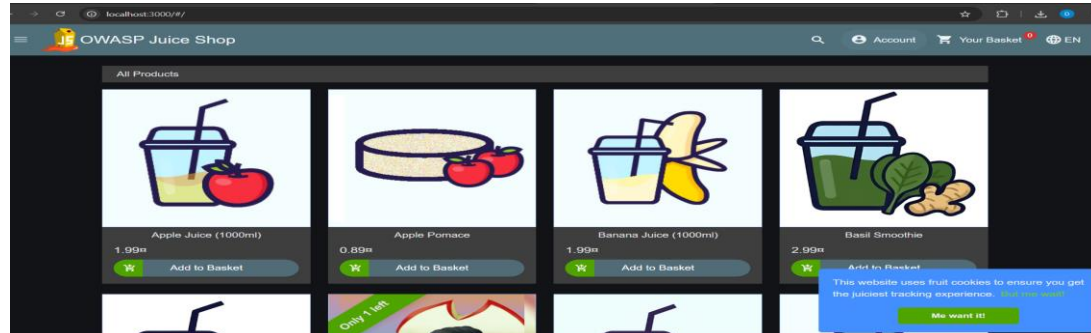
Security impact: Successful exploitation can expose records, alter application data, bypass authentication logic, or disrupt database-backed functionality.

Recommended remediation: Use parameterized queries / ORM-safe query APIs; enforce allowlist validation; separate data from commands; add negative security tests for authentication and search endpoints; log rejected malicious patterns without storing secrets.

Remediation timeline: 0–24 hours: contain and validate. 1–3 days: implement parameterization and tests. 7 days: regression testing and closure evidence.

Retest criteria: Re-run the original test case with the same preconditions. Confirm the attack no longer succeeds, legitimate functionality remains available, and regression coverage prevents recurrence.

Evidence handling: store screenshots and sanitized request/response samples in the security-lab repository; remove passwords, session tokens, API keys and personal information.



8.2 Technical Finding — VAPT-02

Field	Assessment result
Finding	Cross-Site Scripting — DOM / Client-Side XSS
Severity	High
CVSS v3.1	6.1
Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
OWASP mapping	A03:2021 — Injection

Description: Client-side data was capable of reaching a browser execution sink without sufficient contextual output encoding in the training application.

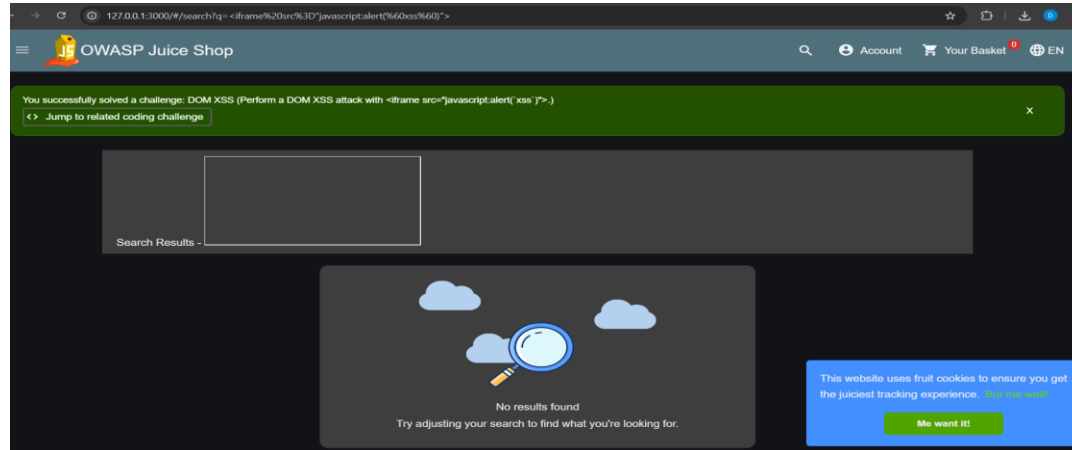
Security impact: A victim who follows an attacker-controlled link or interacts with malicious content may have attacker script execute in the application security context, enabling session abuse or content manipulation.

Recommended remediation: Prefer safe DOM APIs; use context-aware output encoding; sanitize only where required; deploy a restrictive Content Security Policy; avoid unsafe HTML insertion; add automated XSS regression tests.

Remediation timeline: 1–3 days: remove unsafe sinks. 3–7 days: add CSP and tests. 14 days: verify across supported browsers.

Retest criteria: Re-run the original test case with the same preconditions. Confirm the attack no longer succeeds, legitimate functionality remains available, and regression coverage prevents recurrence.

Evidence handling: store screenshots and sanitized request/response samples in the security-lab repository; remove passwords, session tokens, API keys and personal information.



8.3 Technical Finding — VAPT-03

Field	Assessment result
Finding	Broken Access Control / Object-Level Authorization
Severity	Medium
CVSS v3.1	6.3
Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:H/A:N
OWASP mapping	A01:2021 — Broken Access Control

Description: Authorization controls should be enforced server-side for every protected object and action. The lab assessment identified an object-access pattern that can be abused when identifiers are trusted without an ownership/role check.

Security impact: An authenticated low-privilege user may access or modify another user's resource, causing confidentiality and integrity loss.

Recommended remediation: Perform server-side authorization on every request; derive identity from the authenticated session; verify resource ownership/role; use deny-by-default policies; add horizontal and vertical privilege regression tests.

Remediation timeline: 1–3 days: patch authorization checks. 7 days: test all object endpoints. 14 days: complete authorization review.

Retest criteria: Re-run the original test case with the same preconditions. Confirm the attack no longer succeeds, legitimate functionality remains available, and regression coverage prevents recurrence.

Evidence handling: store screenshots and sanitized request/response samples in the security-lab repository; remove passwords, session tokens, API keys and personal information.

OWASP Juice Shop (Express ^4.22.1)

500 Error: Unexpected path: /rest/qwertz

```
at C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\build\routes\angular.js:18:18
at C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\build\lib\utils.js:235:28
at Layer.handle [as handle_request] (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\layer.js:95:5)
at trim_prefix (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:328:13)
at C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:286:9
at router.process_params (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:346:12)
at next (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:280:10)
at C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\build\routes\verify.js:235:5
at Layer.handle [as handle_request] (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\layer.js:95:5)
at trim_prefix (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:328:13)
at C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:286:9
at router.process_params (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:346:12)
at next (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:280:10)
at C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\build\lib\security.js:218:5
at Layer.handle [as handle_request] (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\layer.js:95:5)
at trim_prefix (C:\Users\HP\Downloads\juice-shop-20.2.0_node24_win32_x64\juice-shop_20.2.0\node_modules\express\lib\router\index.js:328:13)
```

8.4 Technical Finding — VAPT-04

Field	Assessment result
Finding	Authentication Weakness / Password Policy
Severity	Critical
CVSS v3.1	9.1
Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
OWASP mapping	A07:2021 — Identification and Authentication Failures

Description: Weak credential controls can make account takeover substantially easier, especially when common passwords, missing rate limits, or predictable recovery behavior are present.

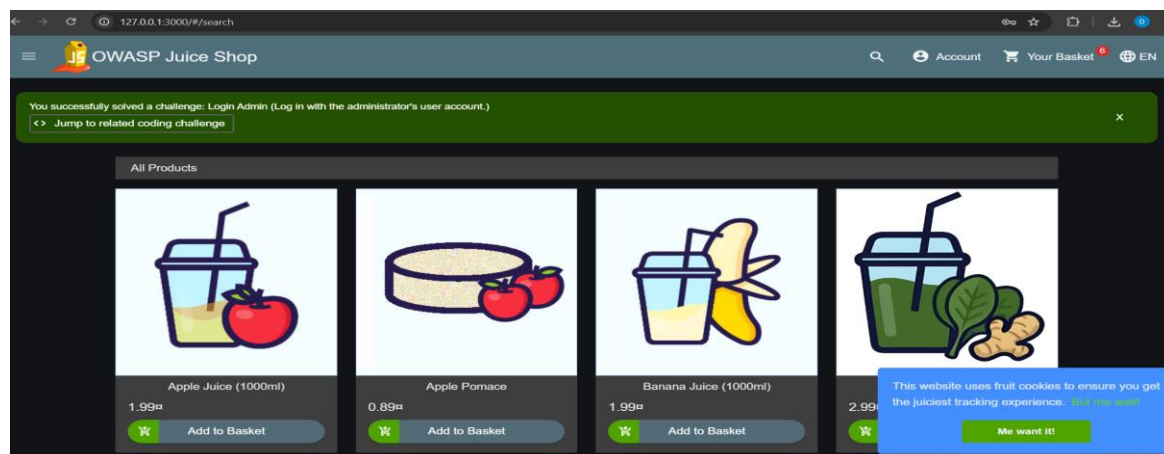
Security impact: Account compromise can expose private data and allow attackers to perform actions as the victim or a privileged account.

Recommended remediation: Enforce strong password screening against common/breached passwords; rate-limit authentication; use secure password hashing; protect recovery flows; add MFA for privileged accounts; monitor repeated failures.

Remediation timeline: 0–24 hours: protect privileged accounts. 1–3 days: strengthen policy and rate limiting. 7–14 days: complete authentication hardening.

Retest criteria: Re-run the original test case with the same preconditions. Confirm the attack no longer succeeds, legitimate functionality remains available, and regression coverage prevents recurrence.

Evidence handling: store screenshots and sanitized request/response samples in the security-lab repository; remove passwords, session tokens, API keys and personal information.



8.5 Technical Finding — VAPT-05

Field	Assessment result
Finding	Sensitive Data Exposure / Secrets Handling
Severity	High
CVSS v3.1	8.2
Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
OWASP mapping	A02:2021 / A05:2021 — Cryptographic Failures / Security Misconfiguration

Description: Sensitive information must not be exposed through source files, backups, logs, client-side bundles, debug responses, or weakly protected endpoints. The assessment included checks for exposed credentials and data leakage patterns.

Security impact: Leaked credentials or sensitive records can enable direct compromise, privacy impact, lateral movement, or fraud.

Recommended remediation: Remove secrets from source control; rotate exposed credentials; use a secrets manager; minimize sensitive logging; encrypt sensitive data in transit and at rest; restrict backup access; verify production error handling.

Remediation timeline: 0–24 hours: rotate any exposed secrets. 1–7 days: clean repositories/logs and implement secret management. 14 days: verify exposure paths.

Retest criteria: Re-run the original test case with the same preconditions. Confirm the attack no longer succeeds, legitimate functionality remains available, and regression coverage prevents recurrence.

Evidence handling: store screenshots and sanitized request/response samples in the security-lab repository; remove passwords, session tokens, API keys and personal information.

Planned Acquisitions

> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year. This will have a significant stock market impact as we will elaborate in detail in the following paragraph:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Our shareholders will be excited. It's true. No fake news.

8.6 Technical Finding — VAPT-06

Field	Assessment result
Finding	Security Misconfiguration / Excessive Information Disclosure
Severity	High
CVSS v3.1	7.3
Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
OWASP mapping	A05:2021 — Security Misconfiguration

Description: Development-oriented settings, verbose error responses, unnecessary interfaces, or exposed metadata can provide attackers with useful information and increase attack surface.

Security impact: Information disclosure can accelerate exploitation, reveal implementation details, or expose unnecessary administrative functionality.

Recommended remediation: Disable debug mode in production; standardize secure headers; remove unused services/routes; minimize server banners; apply secure defaults; review cloud and deployment configuration.

Remediation timeline: 1–3 days: remove debug and unnecessary interfaces. 7 days: baseline secure headers/configuration. 14 days: configuration review.

Retest criteria: Re-run the original test case with the same preconditions. Confirm the attack no longer succeeds, legitimate functionality remains available, and regression coverage prevents recurrence.

Evidence handling: store screenshots and sanitized request/response samples in the security-lab repository; remove passwords, session tokens, API keys and personal information.

```
← → ↺ 127.0.0.1:3000/metrics

# HELP juiceshop_llm_input_tokens_total Number of total input tokens processed
# TYPE juiceshop_llm_input_tokens_total counter
juiceshop_llm_input_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_input_tokens Number of input tokens processed
# TYPE juiceshop_llm_input_tokens counter

# HELP juiceshop_llm_output_tokens_total Number of total output tokens processed
# TYPE juiceshop_llm_output_tokens_total counter
juiceshop_llm_output_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_output_tokens Number of output tokens processed
# TYPE juiceshop_llm_output_tokens counter

# HELP juiceshop_llm_tool_calls_total Number of tool calls made
# TYPE juiceshop_llm_tool_calls_total counter

# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP http_requests_count Total HTTP request count grouped by status code.
# TYPE http_requests_count counter
http_requests_count{status_code="2XX",app="juiceshop"} 101
http_requests_count{status_code="3XX",app="juiceshop"} 356
http_requests_count{status_code="5XX",app="juiceshop"} 1

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 0.1238125
juiceshop_startup_duration_seconds{task="cleanupFtpFolder",app="juiceshop"} 0.315104
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 1.9965434
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 5.2777575
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.0158974
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.0086357
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 5.505

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 7.063

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
```

15. OWASP Coverage & Control Mapping

Finding	OWASP Top 10	Primary defensive control
VAPT-01	A03 Injection	Parameterized queries, allowlist validation, negative security tests
VAPT-02	A03 Injection	Contextual encoding, safe DOM APIs, CSP
VAPT-03	A01 Broken Access Control	Server-side authorization and ownership checks
VAPT-04	A07 Identification & Authentication Failures	Password screening, rate limiting, MFA, secure recovery
VAPT-05	A02 Cryptographic Failures / A05 Misconfiguration	Secrets management, encryption, secure logging/configuration
VAPT-06	A05 Security Misconfiguration	Secure defaults, headers, debug disabled, attack-surface reduction

OWASP Top 10 is a classification framework, not proof that an application is secure. Mature assurance combines it with threat modeling, code review, dependency management, cloud review, monitoring and incident response.

16. Prioritized Remediation Plan

Priority	Action	Owner	Target	Closure evidence
P0	Contain and patch injection paths; review database access controls.	Development + Security	0-3 days	Test results + code diff + retest
P0	Protect privileged accounts and strengthen authentication controls.	Development + IAM	0-3 days	Policy/config evidence + login tests
P1	Rotate exposed credentials/secrets and remove them from code/logs.	DevOps + Security	0-7 days	Rotation record + secret scan
P1	Disable debug/excessive disclosure and harden headers/configuration.	DevOps	0-7 days	Configuration baseline + response capture
P1	Patch XSS sinks and add regression tests/CSP.	Development	0-7 days	Automated test + browser validation
P2	Complete object-level authorization review across all endpoints.	Development + Security	0-14 days	Authorization test matrix

Assign a named owner to each finding, record a closure date, link the change/commit, attach retest evidence and document residual risk when remediation is deferred.

17. Secure Implementation & Validation Work

The capstone includes defensive implementation exercises: password-strength checking, secure-login development, vulnerability scanning and cryptographic practice.

Control area	Implementation lesson
Password security	Screen common/weak passwords and encourage stronger credentials.
Secure login	Use secure credential handling and careful deployment practices.
Vulnerability scanning	Use repeatable discovery and reporting to support remediation.
Cryptography	Use AES-256-GCM, RSA key-pair concepts and HMAC with correct key and nonce management.
Testing discipline	Do not close a vulnerability until the original attack path is retested.

Cryptographic algorithms alone do not make a system secure. Key generation, storage, rotation, nonce/IV handling, authentication tags, access control, error handling and secret management are equally important.

18. Retesting, Metrics & Final Conclusion

Retesting procedure: preserve the original test; apply the remediation; repeat the same attack sequence; confirm the exploit is blocked; run regression tests; capture sanitized evidence; and formally close only after review.

Metric	Target
Critical findings overdue	0
High findings older than SLA	0
Authentication security tests passing	100%
Secrets detected in repository	0
Authorization coverage	100% of externally reachable object endpoints
Critical findings successfully retested	100% before closure

Conclusion: the capstone demonstrates a structured vulnerability-assessment workflow and produces actionable remediation guidance. Production decisions should be based on a fresh assessment of the actual target environment.

19. GitHub Security Lab Repository Structure

Recommended structure:

```
vapt-capstone/  
├── README.md  
├── reports/  
│   └── Drushya_ND_Final_VAPT_Penetration_Testing_Report.pdf  
├── findings/  
│   ├── VAPT-01-injection.md  
│   ├── VAPT-02-xss.md  
│   ├── VAPT-03-access-control.md  
│   ├── VAPT-04-authentication.md  
│   ├── VAPT-05-sensitive-data.md  
│   └── VAPT-06-misconfiguration.md  
├── evidence/  
│   ├── screenshots/  
│   └── sanitized-requests/  
├── remediation/  
│   └── remediation-plan.md  
└── docs/  
    └── scope-and-methodology.md
```

Repository hygiene: do not commit passwords, cookies, session tokens, API keys, private keys, personal records or unredacted production logs.

20. References & Standards

OWASP Foundation — OWASP Juice Shop: <https://owasp.org/www-project-juice-shop/>

OWASP Foundation — OWASP Top 10:2021: <https://owasp.org/Top10/2021/>

OWASP Foundation — Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>

FIRST — CVSS v3.1: <https://www.first.org/cvss/v3.1/>

Submission note: attach sanitized screenshots/evidence to the repository and replace generic evidence descriptions with exact lab screenshots where required.